UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Faculty of Computing

SECP3623

Database Programming

2025/2026 SEMESTER 1

Assignment 2 (10%) – Transaction Management

Name	:	Lau Yee Wen
Matric Id	:	A23CS0099
Section	:	Section 01

Name	:	Lee Yin Shen
Matric Id	:	A23CS0236
Section	:	Section 01

Name	:	Poh Lok Yee
Matric Id	:	A23CS0262
Section	:	Section 01

Instructions:

This assignment evaluates your ability to apply the concept of serializability, differentiate between serial and non-serial transaction schedules and implement locking techniques to manage simultaneous transactions.

Academic Integrity

All students must work in a group of **TWO(2)**. The use of AI-generated tools or sharing of answers between students **is strictly prohibited** and will result in **zero marks**. You are required to complete the answers in **handwritten form**. Please take **clear screenshots** of your answers and submit them in **PDF format**.

QUESTION 1 (15 MARKS)

Consider the schedule transactions in Table 1 and answer all following questions (i) – (iv).

Table 1: Schedule T1, T2 and T3

Time	T1	T2	T3
t_1	begin transaction		
t_2	read_item(X)		
t_3	read_item(Y)	begin transaction	
t_4	commit	read_item(M)	begin transaction
t_5		read_item(N)	read_item(N)
t_6		M=(N+M)*10	read_item(M)
t_7		write_item(M)	N=N+150
t_8		commit	write_item(N)
t_9			commit

i) Draw the precedence graph for the transaction schedule in Table 1 to check for serializability.

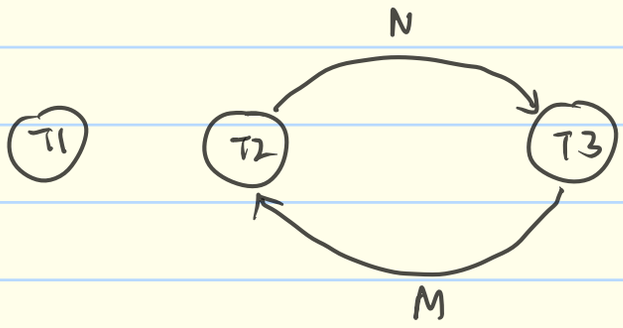
ii) Explain the precedence graph produced in (i).

iii) Apply the locking technique to the transaction schedule in Table 1, and revise the transaction schedule in Table 1 using the following format:

T1	T2	T3

iv) Identify the problem(s) that arise in (iii) and then explain any potential problems that may still occur after applying the locking technique in concurrency control based on your answer in (iii).

i)



ii) There is no edges for T1 because it only access the item X and Y, which is not conflict with T2 and T3

There is an edge from T2 to T3 with item N because T3 writes N after T2 read it.

There is an edge from T3 to T2 with item M because T2 writes M after T3 read it.

The schedule is non-conflict serializable.

iii)

Time	T1	T2	T3
t1	begin_transaction		
t2	S - lock (X)		
t3	read_item (X)	begin_transaction	
t4	unlock_item (X)	X - lock (M)	begin_transaction
t5	S _lock (Y)	read_item (M)	X - lock (N)
t6	read_item (Y)	S - lock (N)	read_item (N)
t7	unlock_item (Y)	wait	S - lock (M)
t8	commit	wait	wait

iv) The problem is deadlock. This is because T2 is holding item M, and it is waiting for N from T3. While T3 is holding item N, and it is waiting for M from T2. This become a cycle of waiting.

The other problem that may still occur is called starvation. This is because to fix deadlock, we need to abort one transaction. For example if abort T2, T3 will be successful to run, but T2 will never finish its works. This is called starvation.

QUESTION 2 (15 MARKS)

Based on the transaction schedule in Table 2, answer all the following questions:

Table 2: Transaction Schedule for T4-T5-T6

	T4	T5	T6
	begin transaction		
	read_item(P)		
	read_item(Q)	begin transaction	
↓	P=(P*5)+Q	read_item(Q)	
time	write_item(P)	read_item(R)	begin transaction
	commit	Q=Q+R	read_item(R)
		write_item(Q)	R=R-100
		commit	write_item(R)
			commit

i) Draw the precedence graph for the transaction schedule in Table 2 to check for serializability.

ii) Explain the precedence graph produced in (i).

iii) Apply the two-phase locking (2PL) protocol to emphasize the positioning of lock and unlock operations in each transaction using the following format.

T4	T5	T6

i)

ii) There is an edge from T4 to T5 with item Q because T5 writes Q after T4 read it.
There is an edge from T5 to T6 with item R because T6 writes R after T5 read it.
This schedule is conflict serializable.

iii)

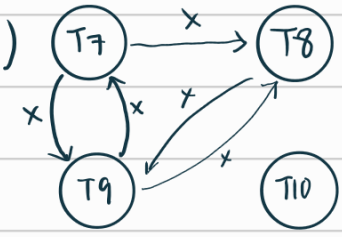
Time	T4	T5	T6
t_1	begin_transaction		
t_2	write_lock (P)		
t_3	read_item (P)	begin_transaction	
t_4	read_lock (Q)	write_lock (Q)	
t_5	read_item (Q)	wait	begin_transaction
t_6	P = (P*5) + Q	wait	write_lock (R)
t_7	write_item (P)	wait	read_item (R)
t_8	commit / unlock (P,Q)	wait	R = R-100
t_9		read_item (Q)	write_item (R)
t_{10}		read_lock (R)	commit / unlock (R)
t_{11}		read_item (R)	
t_{12}		Q = Q + R	
t_{13}		write_item (Q)	
t_{14}		commit / unlock (Q,R)	

Question 3

a)

i) Non-serial schedule. T7, T8, T9 and T10 are interleaving, more than one transaction is active at the same time and their operation is overlap.

ii)



iii) Non-Conflit Serializable Schedule as it have a cycle, the schedule cannot transformed to any serial schedule.

iv) Lost update will cause since T9 reads X before T7 write. T9 read the value at t_3 but T7 write the new value of X on t_4, T9 is reading uncommited data that causing T7's update to be lost.

b) i)

Time	T11	T12	T13	T14
t_1	Begin Transaction			
t_2	x-lock (A)	Begin Transaction		
t_3	read (A)	x-lock (B)	Begin Transaction	
t_4		read (B)	request x-lock (A)	Begin Transaction
t_5	A = A+200	B = B-50	WAIT	x-lock (C)
t_6	write (A)		WAIT	read (C)
t_7	unlock (A)	write (B)	WAIT	C = C+10
t_8		unlock (B)	x-lock (A)	write (C)
t_9			read (A)	unlock (C)
t_{10}	commit			
t_{11}		request x-lock (A)	A = A-30	
t_{12}		WAIT	write (A)	
t_{13}		WAIT	unlock (A)	
t_{14}		x-lock (A)		commit
t_{15}		read (A)		
t_{16}		A = A*2		
t_{17}		write (A)		
t_{18}		commit / unlock (A)	commit	

ii) The issue arise is starvation on transaction T12. It force to wait for A while T11 and T13 served first. T13 are just normal wait but T12 repeatedly bypassed by other transactions, resulting in unfair long delay compared to its arrival time.

iii)

Time	T11	T12	T13	T14
t_1	Begin Transaction			
t_2	write_lock (A)	Begin Transaction		
t_3	read (A)	x-lock (B)	Begin Transaction	
t_4		read (B)	WAIT	Begin Transaction
t_5	A = A+200	B = B-50	WAIT	write_lock (C)
t_6	write (A)		WAIT	read (C)
t_7		write (B)	WAIT	C = C+10
t_8			WAIT	write (C)
t_9	commit/unlock (A)		write_lock (A)	
t_{10}		write_lock (A)	read (A)	
t_{11}		WAIT		
t_{12}		WAIT	A = A-30	
t_{13}		WAIT	write (A)	commit/unlock (C)
t_{14}		WAIT		
t_{15}		WAIT		
t_{16}		WAIT		
t_{17}		WAIT		
t_{18}		WAIT	commit/unlock (A)	
t_{19}		read (A)		
t_{20}		A = A*2		
t_{21}		write (A)		
t_{22}		commit/unlock (A,B)		

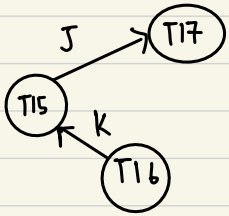
iv) There will be the same starvation problem of T12 as basic locking.

The another problem are system will slower since transaction are serialized, transaction hold locks from start to commit for a long time.

4. (i) Non-serial schedule.

A serial schedule requires one transaction to complete fully before starting the next one.
In table 4, transactions T15, T16, T17 overlap and interleave in execution.
For example, T16 and T17 begin before T15 commits. Hence, the schedule is non-serial.

(ii)



(iii) Conflic-serializable schedule, because its precedence graph has no cycle.

(iv) Lost Update problem. At t_4, T15 writes J. At the same time, T17 reads J. However, T15 has not committed yet. T17 is reading uncommitted data.
This shows multiple transactions update the same data items (J and K) without proper synchronization. As a result, updates from one transaction may overwrite updates from another, causing loss of data consistency.

(v)

(v).

Table 4.1: Schedule T15, T16 and T17

Time	T15	T16	T17
t_1	begin transaction		
t_2	read (J)	begin transaction	
t_3	J = J + 10	read (K)	begin transaction
t_4	write (J)	K = K - 5	read (J)
t_5	Read (K)	write (K)	J = J - 15
t_6	K = K + 20		write (J)
t_7	write (K)	commit	
t_8	commit		commit

(v)

Time	T_{15}	T_{16}	T_{17}
t_1	begin_transaction		
t_2	write_lock (J)	begin_transaction	
t_3	read (J)	write_lock (K)	begin_transaction
t_4	J = J + 10	read (K)	write_lock (J)
t_5	write (J)	K = K - 5	WAIT
t_6	write_lock (K)	write (K)	WAIT
t_7	WAIT	commit / unlock (K)	WAIT
t_8	read (K)		WAIT
t_9	K = K + 20		WAIT
t_{10}	write (K)		WAIT
t_{11}	commit / unlock (J, K)		WAIT
t_{12}			read (J)
t_{13}			J = J - 15
t_{14}			write (J)
t_{15}			commit / unlock (J)
t_{16}			